

The Stone House

Arben a moștenit un teren frumos pe dealurile care domină orașul Gjirokaștër. Inspirat de faimoasele case cu acoperiș de piatră ale orașului, el plănuiește să își construiască propria casă tradițională.

Terenul este reprezentat ca o grilă pătratică $N \times N$, în care liniile și coloanele sunt numerotate de la 1 la N . Terenul stâncos are o particularitate: există exact un bolovan mare, imposibil de mutat, în fiecare linie și în fiecare coloană a grilei.

Pentru a pune fundația casei sale, Arben trebuie să selecteze o **subgrilă pătratică goală** — o regiune de dimensiune $S \times S$ care nu conține niciun bolovan.

Fie M numărul întreg maxim astfel încât, pentru *orice* aranjare a bolovanilor care satisface condițiile de mai sus, se garantează că există o subgrilă pătratică goală de dimensiune $M \times M$. Arben dorește să găsească *orice* subgrilă pătratică goală de dimensiune cel puțin $M \times M$.

Pentru a cerceta terenul, Arben îi cere ajutorul prietenei sale Besa. Besa a adus un dispozitiv special de măsurare numit **Gurëmatës** („numărătorul de pietre”).

Într-o singură operație, Besa poate configura Gurëmatës-ul pentru a scana o subgrilă dreptunghiulară dată prin colțul său din stânga-sus (x_1, y_1) și colțul său din dreapta-jos (x_2, y_2) . Dispozitivul returnează numărul exact de bolovani din acea regiune. Gurëmatës-ul este alimentat de o baterie mică și fiecare scanare consumă multă energie, așa că Besa trebuie să îl folosească de cât mai puține ori posibil.

Scrieți un program care folosește Gurëmatës-ul pentru a localiza o subgrilă pătratică goală de dimensiune $S \times S$ cu $S \geq M$ și raportează poziția acesteia.

Detalii de implementare

Trebuie să implementați următoarea funcție:

```
std::vector < int > solve(int N);
```

- N : dimensiunea grilei.
- Funcția trebuie să returneze exact patru numere întregi $\{x_1, y_1, x_2, y_2\}$: colțul din stânga-sus și colțul din dreapta-jos ale subgrilei pătratice goale pe care a găsit-o. Trebuie să fie

îndeplinită condiția $x_2 - x_1 = y_2 - y_1 = S - 1$ cu $S \geq M$.

- Această funcție se apelează exact o singură dată pentru fiecare caz de test.

Funcția poate face apeluri către următoarea funcție:

```
int ask(int x1, int y1, int x2, int y2);
```

- x_1, y_1 : linia și coloana colțului din stânga-sus al subgrilei scanate.
- x_2, y_2 : linia și coloana colțului din dreapta-jos al subgrilei scanate.
- Trebuie să fie îndeplinite condițiile $1 \leq x_1 \leq x_2 \leq N$ și $1 \leq y_1 \leq y_2 \leq N$. În caz contrar, programul vostru va fi oprit imediat și va primi 0 puncte pentru acel caz de test.
- Funcția returnează numărul de bolovani din subgrila scanată.

Grader-ul nu este adaptiv: pozițiile bolovanilor sunt fixate înainte de apelul funcției `solve` și nu depind de apelurile tale la `ask`.

Restricții

- $2 \leq N \leq 500$
- Poți face cel mult $2.5 \cdot 10^5$ apeluri la `ask`.

Subtask-uri

Subtask	Puncte	Constrângeri suplimentare
1	15	$N \leq 40$
2	25	$N \leq 150$
3	60	Fără constrângeri suplimentare.

Punctaj

Dacă `solve` returnează o regiune invalidă, face un apel invalid la `ask` sau apelează `ask` de mai mult de $2.5 \cdot 10^5$ ori, veți primi 0 puncte pentru acel caz de test.

În caz contrar, fie Q numărul de apeluri la `ask`. Pentru acel caz de test veți primi punctajul subtask-ului înmulțit cu $\min(1, f(Q))$, unde

$$f(Q) = \begin{cases} \max(0, -0.05 \cdot Q + 1.55), & \text{if } Q \leq 15 \\ \max(0, -0.19 \cdot \log_{10}(Q) + 1.02), & \text{if } Q > 15 \end{cases}$$

Punctajul tău pentru un subtask este punctajul minim obținut pe toate cazurile de test din acel subtask. Punctajul final este suma punctajelor subtask-urilor.

Ca să vă ajute să estimați scorul mai bine, tabelul de mai jos arată multiplicatorul aproximativ pentru mai multe variante ale lui Q :

Numărul de interogări (Q)	Multiplicator ($\approx$)
$1 \leq Q \leq 11$	1.00
12	0.95
15	0.80
50	0.70
100	0.64
1000	0.45
10000	0.26
100000	0.07
$Q \geq 233573$	0.00

Grader de probă

Grader-ul de probă citește datele de intrare în următorul format:

```
N
c_1 c_2 ... c_N
```

Aici, c_i este coloana bolovanului din linia i ($1 \leq c_i \leq N$).

Grader-ul de probă afișează o singură linie care conține multiplicatorul $\min(1, f(Q))$ obținut de întrebările tale, sau 0 dacă subgrila returnată sau vreun apel la `ask` este invalid.

Exemple

Exemplul 1

Datele de intrare pentru grader-ul de probă:

```
4
1 2 3 4
```

Bolovanii se află la $(1, 1)$, $(2, 2)$, $(3, 3)$ și $(4, 4)$. Programul tău nu știe acest lucru.

Grader-ul apelează `solve(4)`. O posibilă secvență de apeluri este:

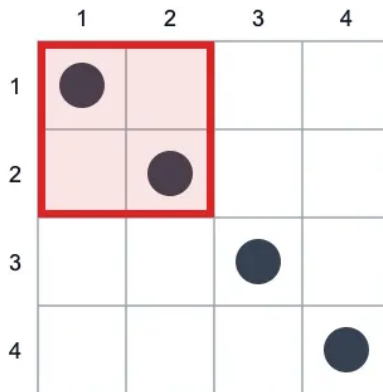
- `ask(1, 1, 2, 2)` returnează 2.
- `ask(2, 4, 4, 4)` returnează 1.
- `ask(3, 1, 3, 2)` returnează 0.

Apoi `solve` returnează $\{1, 3, 2, 4\}$.

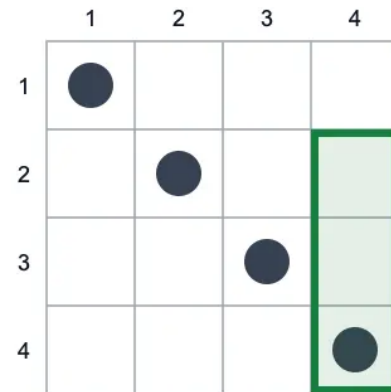
Datele de ieșire ale grader-ului de probă:

```
1
```

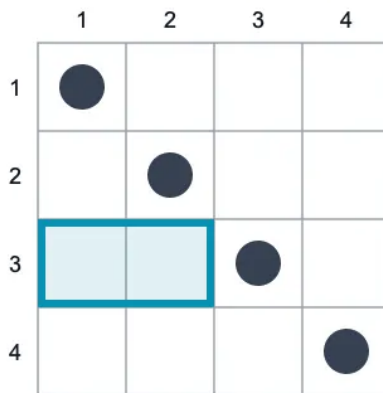
Primul apel scanează blocul 2×2 din stânga-sus și găsește bolovanii de la $(1, 1)$ și $(2, 2)$. Al doilea scanează liniile de la 2 la 4 ale coloanei 4 și găsește bolovanul de la $(4, 4)$. Al treilea scanează primele două celule ale liniei 3 și nu găsește niciun bolovan. Regiunea returnată acoperă liniile de la 1 la 2 și coloanele de la 3 la 4, ceea ce reprezintă o subgrilă goală 2×2 .



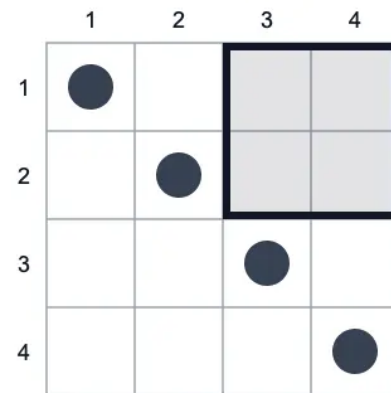
ask(1, 1, 2, 2) returns 2



ask(2, 4, 4, 4) returns 1



ask(3, 1, 3, 2) returns 0



return {1, 3, 2, 4}

Pentru $N = 4$, programul a folosit $Q = 3$ apeluri, deci $f(Q) = 1.4$ și multiplicatorul este 1. Sunt acceptate și alte răspunsuri, de exemplu $\{3, 1, 4, 2\}$, $\{1, 2, 1, 2\}$, $\{3, 2, 3, 2\}$, $\{3, 4, 3, 4\}$ sau orice celulă goală, deoarece $M = 1$ pentru $N = 4$.